S09 – Recon Workflow & File Upload Exploitation

Name: MUHAMMAD ZAIN NASIR

Cohort / Programme: Atomcamp – Digital Forensics & Cyber Security

Submitted: November 2026

Repository: https://github.com/the-raven-zn/dfcs-bootcamp/tree/main/s09-recon-and-upload

Target: DVWA v1.0.7, bundled on a Metasploitable2 VM – Target IP `192.168.206.130` (confirmed via `ifconfig` on the target machine – differs from the handout's example IP of `192.168.50.129`; every command in this report uses the real, confirmed address)

Attacker machine: Kali Linux (VM)

Scope: Isolated host-only lab network. No systems outside this lab were tested.

Contents: Part A (Recon Workflow, 10 stages) · Part B (DVWA File Upload, all levels) · Part C (Non-Technical Remediation Brief) · References

Method. I kept the handout's ten-step recon structure but made deliberate changes, justified in Part A, rather than copying it verbatim – most notably adding an explicit host-discovery step and honestly skipping subdomain enumeration and WAF detection, which do not apply to a bare internal IP. Part B exploits DVWA's File Upload vulnerability at every security level this build actually ships (Low, Medium, High), bypasses each control in turn, and chains the High-level bypass to a second vulnerability (File Inclusion) to achieve real code execution. Part C restates the most serious finding for a non-technical audience.

Part A – My Own Recon Workflow

I kept the handout's ten-step structure but made deliberate changes, justified below, rather than copying it verbatim.

#	Stage	Question answered	Tool(s) used	Output	Stopping condition
1	Host discovery	What's alive on my subnet?	`nmap -sn 192.168.206.0/24`	4 live hosts: gateway (.1), Kali, target (.130), one unknown host (.128)	Every responsive host on /24 listed
2	Connectivity check	Is the target reachable from Kali?	`ping -c 3 $TARGET`	0% packet loss	3 successful replies
3	Full port scan	What services are exposed?	`nmap $TARGET`	20 open ports (ftp, ssh, telnet, smtp, dns, http, rpcbind, samba, rexec family, rmiregistry, ingreslock, nfs, ccproxy-ftp, mysql, postgresql, vnc, X11, irc, ajp13, unknown 8180)	Scan completes, all ports classified
4	Web-port focus	Any alternate web/TLS listeners?	`nmap -p 80,8080,8081,443,8443,4443 $TARGET`	Only port 80 open; 443, 4443, 8080, 8081, 8443 all closed	All 6 ports classified
5	HTTP headers	What does the server reveal about itself?	`curl -I http://$TARGET`	Apache/2.2.8 (Ubuntu) DAV/2, PHP/5.2.4-2ubuntu5.10 – both long past end-of-life	Headers printed

#	Stage	Question answered	Tool(s) used	Output	Stopping condition
6	Technology fingerprinting	What is it actually built with?	`whatweb -v $TARGET`	Confirms Apache 2.2.8, PHP 5.2.4, WebDAV v2 enabled, title “Metasploitable2 - Linux”	Structured summary returned
7	Directory brute force	What’s reachable, and does login gate it?	`gobuster dir -u http://$TARGET/dvwa -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak`	Only pre-login paths found (login.php, about.php, robots.txt, README.txt, setup.php); everything else 302-redirects to login	Wordlist exhausted (18,452/18,452)
8	Misconfiguration / vuln scan	What’s outdated or misconfigured?	`nikto -h http://$TARGET/dvwa`	Directory indexing on /config/ and /docs/, missing security headers, HTTP TRACE enabled, PHP Easter Egg disclosure, outdated Apache/PHP confirmed again	8231 requests, scan completes
9	TLS posture	Is there a TLS service to assess at all?	`sslscan $TARGET:80`	No TLS listener on port 80 – correctly reported as a non-finding, not a “secure” finding	Scan returns no protocol/cipher data
10	Subdomain enumeration / WAF	Not applicable to a bare internal IP	– Skipped, justified below	N/A	–

Deviations from the Session's Workflow, and Why

- **Added an explicit host-discovery step** (`nmap -sn`) before targeting a single IP. This was necessary because the target's real IP (192.168.206.130) differs from the handout's example (192.168.50.129), and it let me confirm the gateway, my own Kali host, and one additional live host on the same subnet before committing to a target.
- **Skipped subdomain enumeration and WAF detection as literally written.** Both require a real domain name or a public internet-facing target. 192.168.206.130 is a private lab IP with no DNS records at all; running subfinder/wafw00f against it is the wrong tool for this kind of target, not a failed test. I chose to document this honestly rather than run those tools against an unrelated public domain just to produce output.
- **Skipped katana and ffuf** once gobuster had already produced a path inventory of the pre-login surface. Running a second, largely overlapping URL-discovery tool against a site that mostly redirects unauthenticated requests to login.php would have reproduced the same limited information. nikto and sslscan were prioritized instead because they test categories gobuster cannot reach (server misconfiguration and TLS posture).

Notable Finding from Part A

Gobuster found almost none of the real application surface – `/vulnerabilities/` and `/hackable/` never appeared in the unauthenticated scan, because every protected DVWA path 302-redirects to login.php without a valid session. This is a legitimate, reportable limitation: an external, unauthenticated scan of this application significantly understates its real attack surface. All vulnerability discovery and exploitation in Part B required an authenticated session throughout.

Part A – Evidence

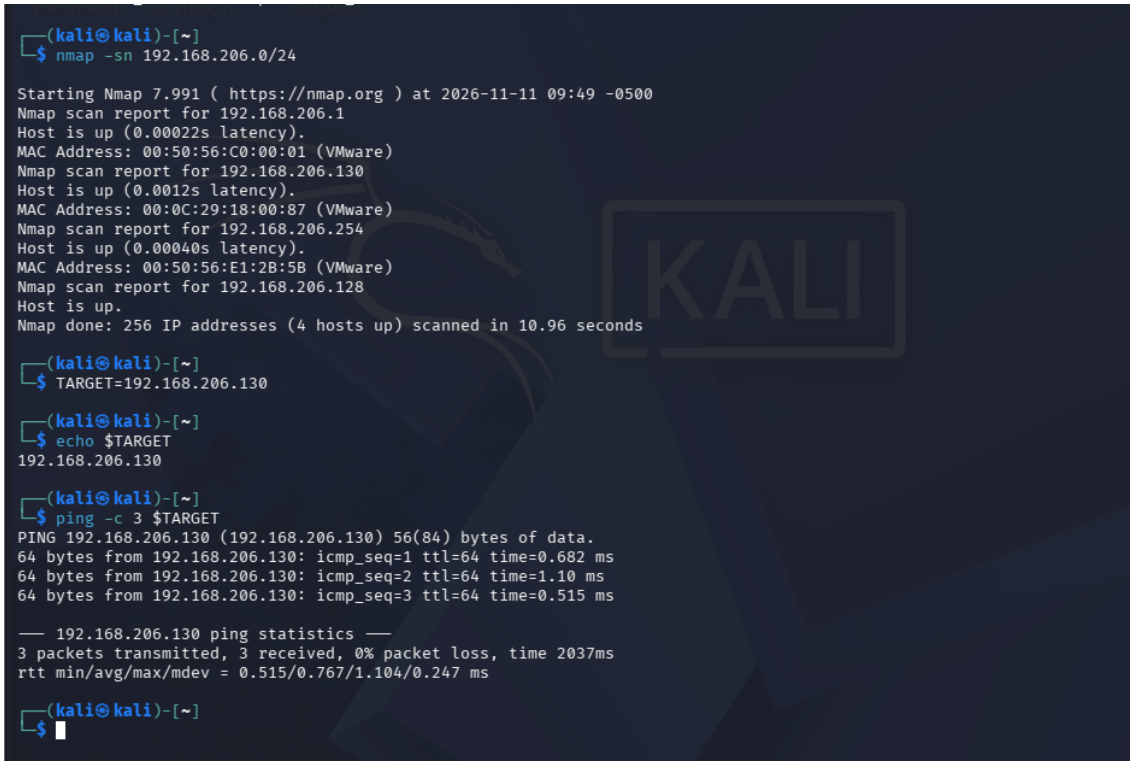


Figure 1: Host discovery and connectivity confirmation

```
┌──(kali㉿kali)-[~]
└─$ nmap $TARGET
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-11 09:51 -0500
Nmap scan report for 192.168.206.130
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8009/tcp open  ajp13
8180/tcp open  unknown
MAC Address: 00:0C:29:18:00:87 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 4.84 seconds

┌──(kali㉿kali)-[~]
└─$
```

Figure 2: Full port scan – 20 open ports

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 80,8080,8081,443,8443,4443 $TARGET
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-11 09:52 -0500
Nmap scan report for 192.168.206.130
Host is up (0.00085s latency).

PORT     STATE  SERVICE
80/tcp   open   http
443/tcp  closed https
4443/tcp closed pharos
8080/tcp closed http-proxy
8081/tcp closed blackice-icecap
8443/tcp closed https-alt
MAC Address: 00:0C:29:18:00:87 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 4.66 seconds

┌──(kali㉿kali)-[~]
└─$
```

Figure 3: Web/TLS port focus scan – only port 80 open

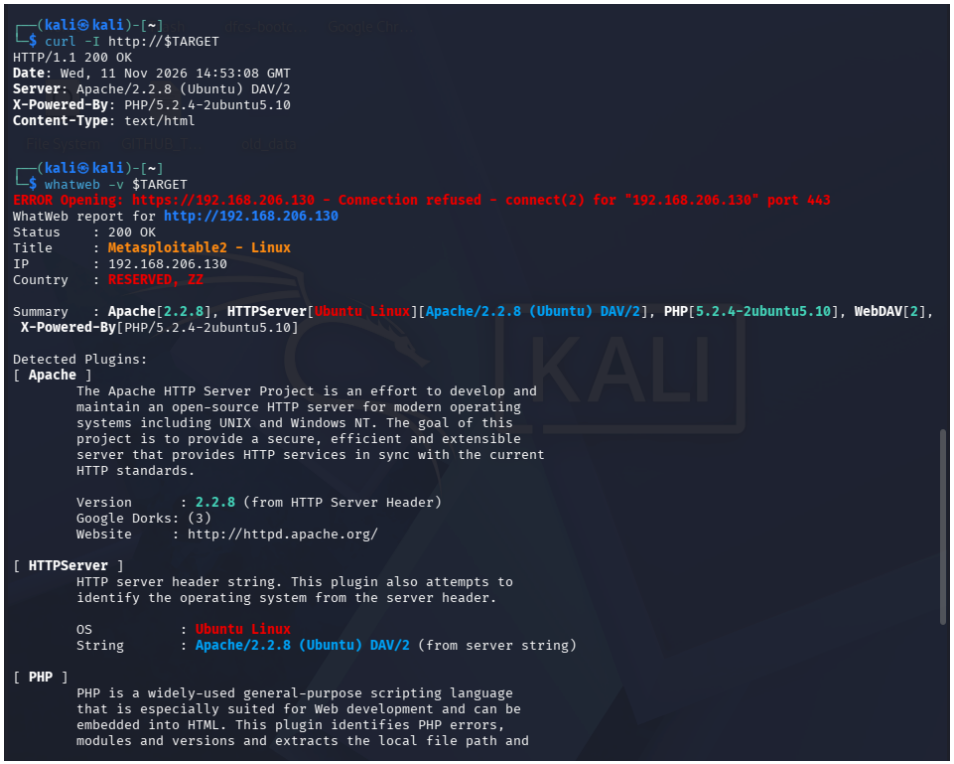


Figure 4: HTTP headers via curl, and whatweb fingerprinting (part 1)



Figure 5: whatweb fingerprinting (part 2)

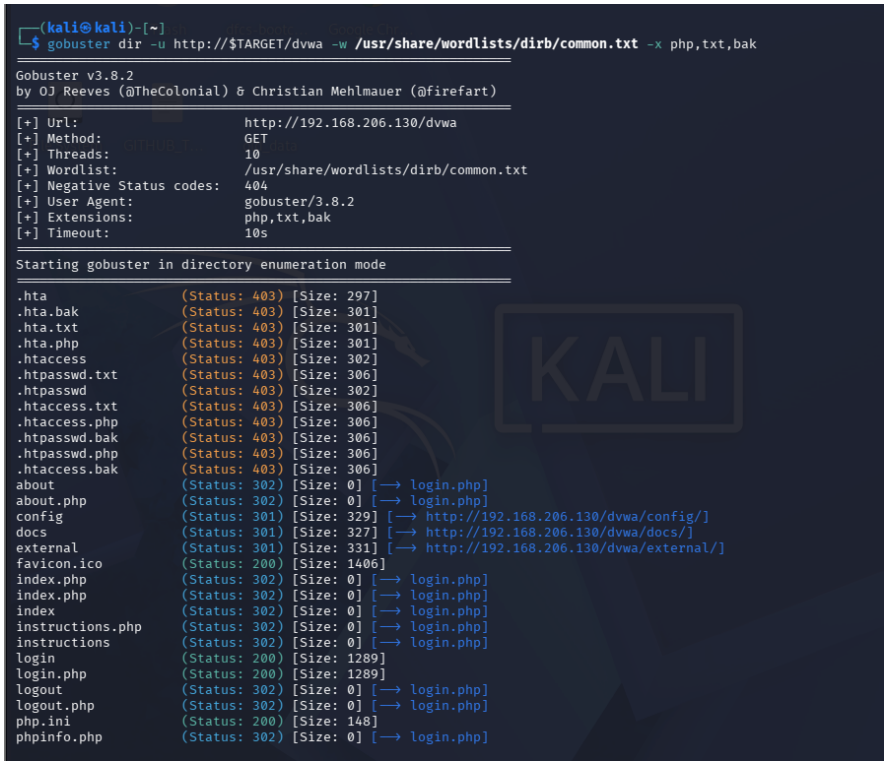


Figure 6: gobuster directory brute force against /dvwa (part 1)

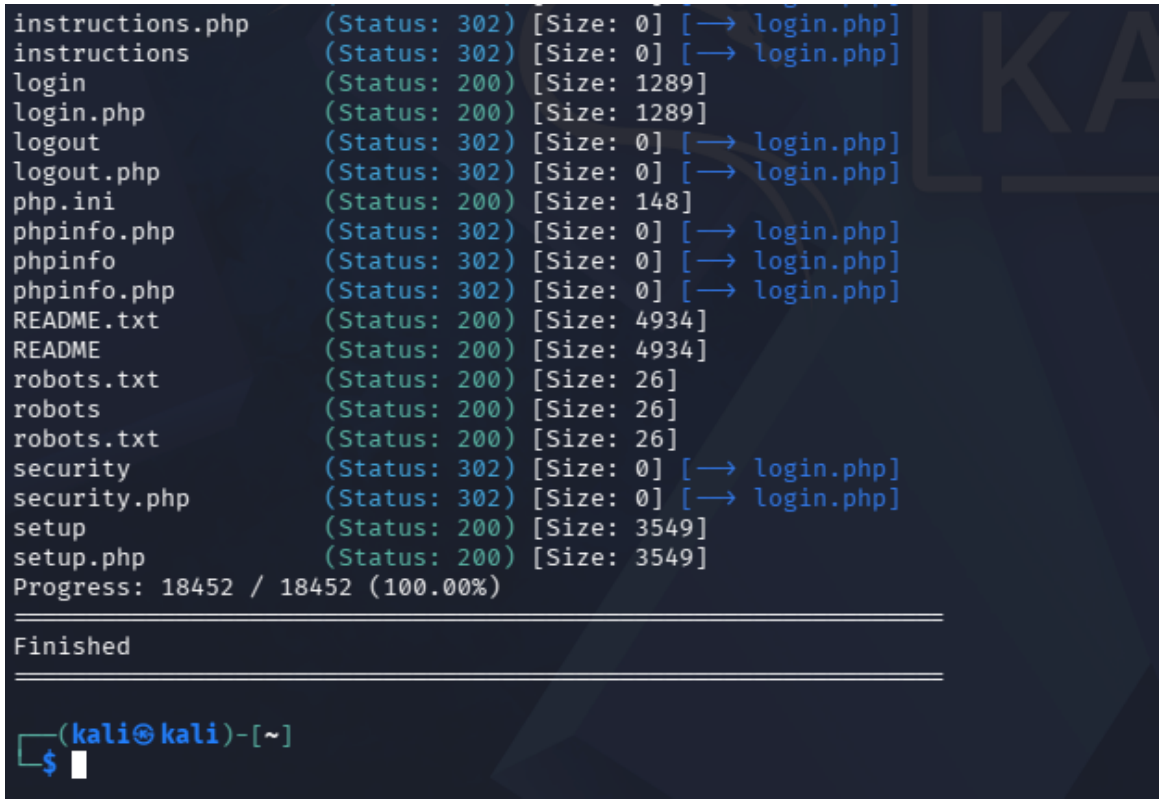


Figure 7: gobuster directory brute force against /dvwa (part 2)

Part B – DVWA File Upload, All Levels

Scope note: the installed DVWA is **v1.0.7** (confirmed in the page footer, visible in the security-level screenshots below), which ships only **Low / Medium / High** security levels. The **Impossible** level described in the handout does not exist in this build – confirmed directly by expanding the security-level dropdown. This is documented honestly below, with a substitute source-code gap analysis in its place.

Baseline (pre-exploitation)

Security level confirmed Low, then a legitimate .jpg was uploaded to establish that the upload mechanism works and to learn the storage path before attempting anything malicious.

Low – Unrestricted Upload

A minimal PHP web shell was created and uploaded with zero server-side validation, then used to run four diagnostic commands.

```
&lt;?php if(isset($_GET['cmd'])){ system($_GET['cmd']); } ?&gt;
```

Privilege level obtained: `uid=33(www-data)` – the Apache service account, not root. Severity remains critical (arbitrary command execution, and read access to configuration files that typically hold database credentials), but the actual privilege level is stated accurately here rather than overstated.

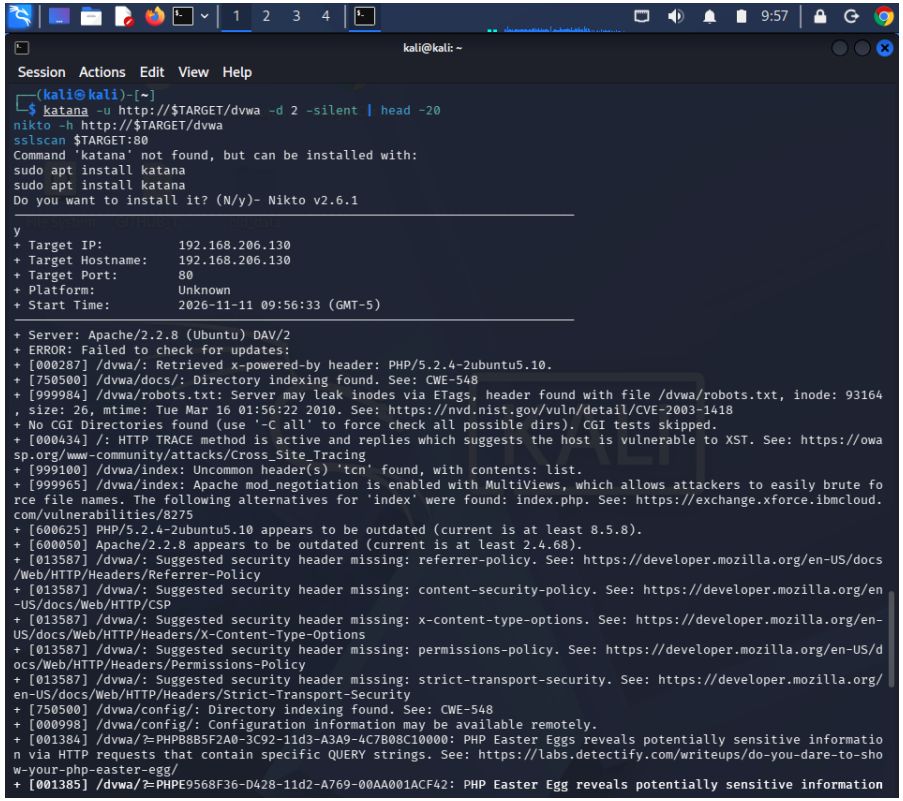


Figure 8: nikto misconfiguration scan (part 1)

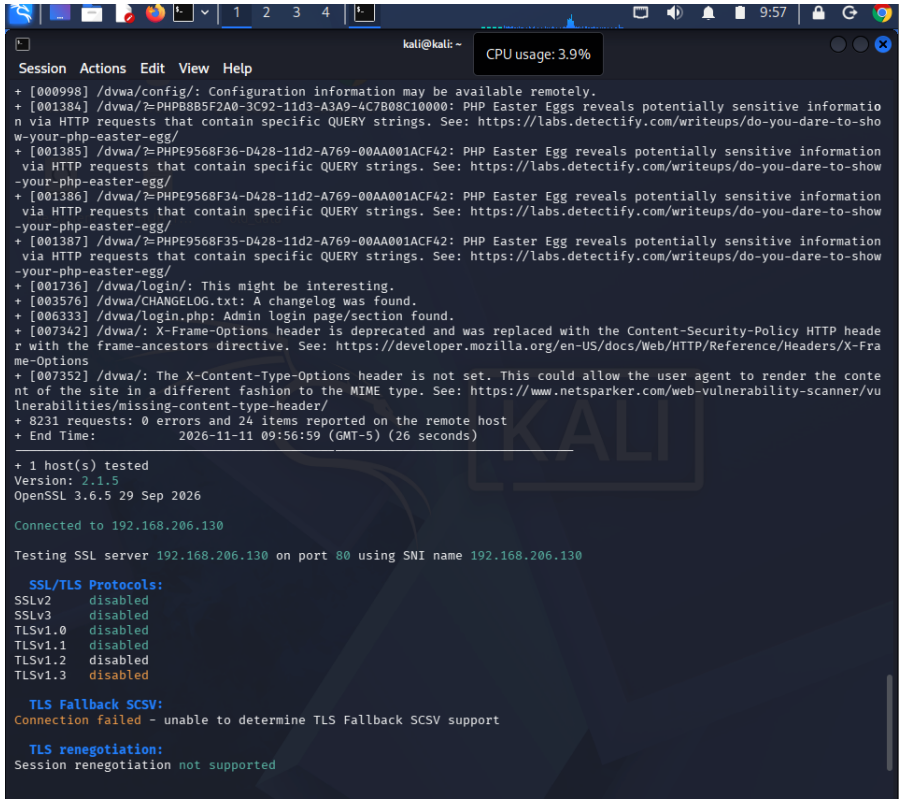


Figure 9: nikto scan (part 2) and start of sslscan

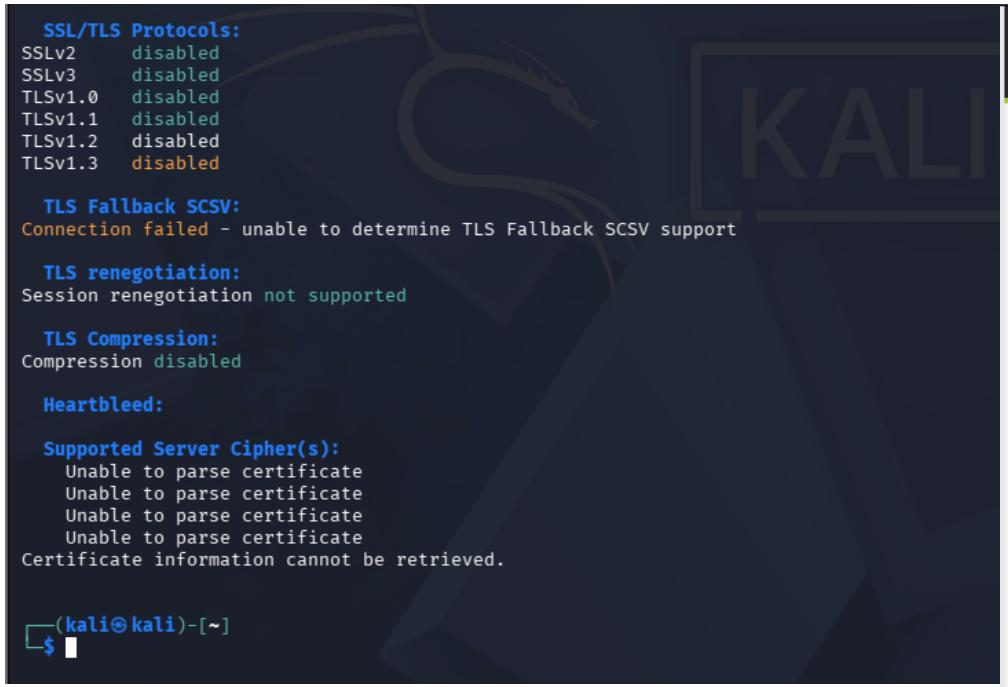


Figure 10: sslscan against port 80 – no TLS service present

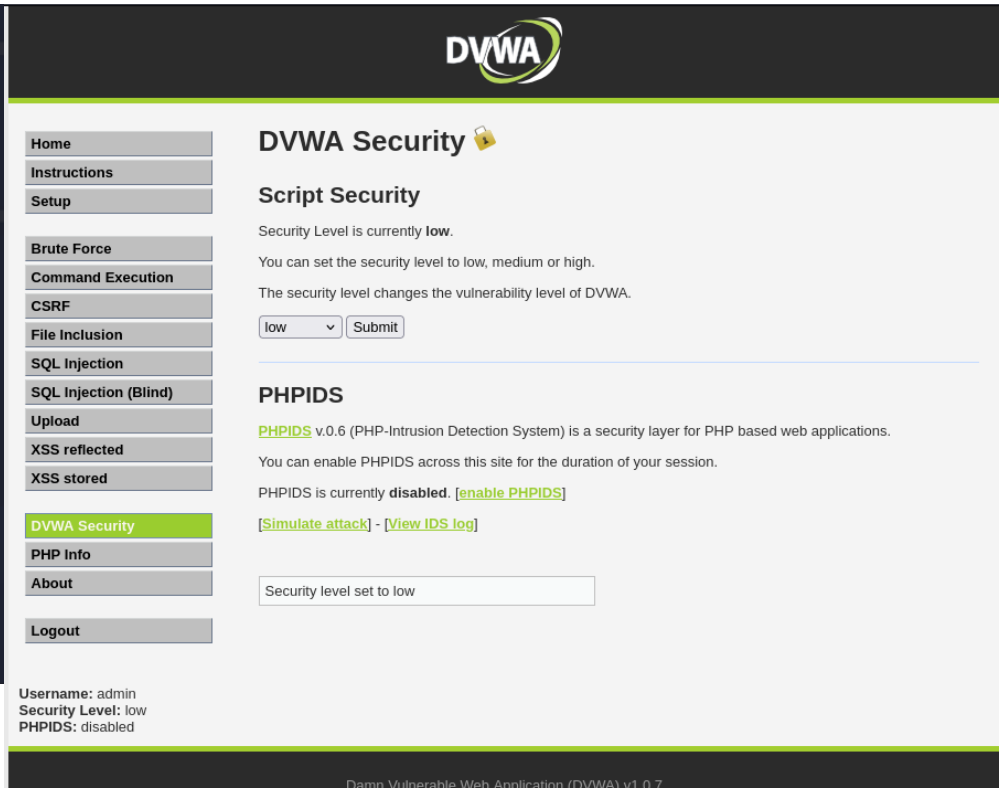


Figure 11: Security level confirmed: Low

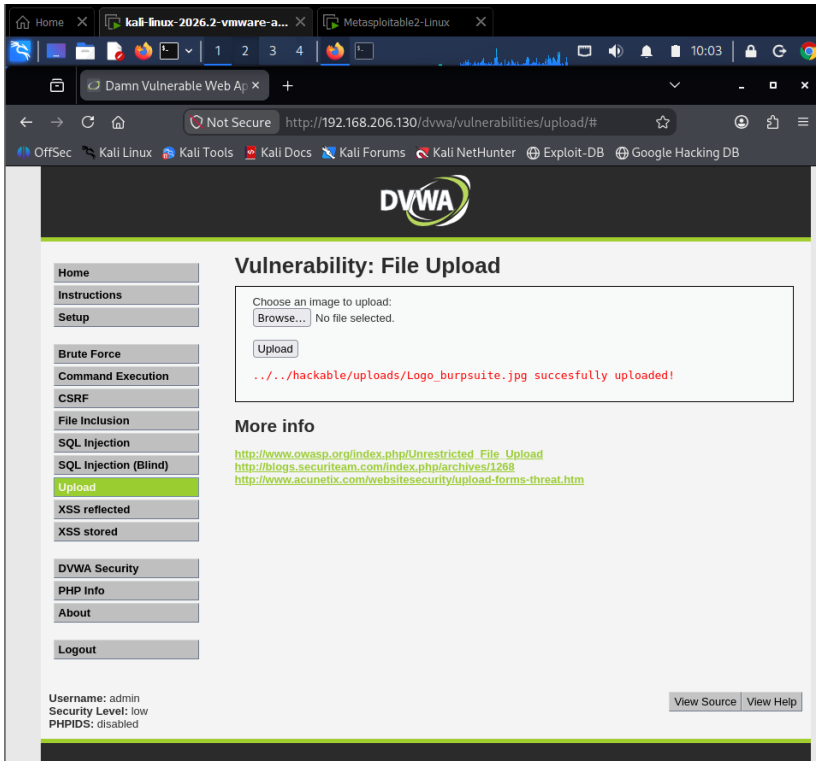


Figure 12: Baseline legitimate image upload succeeds, revealing the storage path



Figure 13: shell.php created on Kali

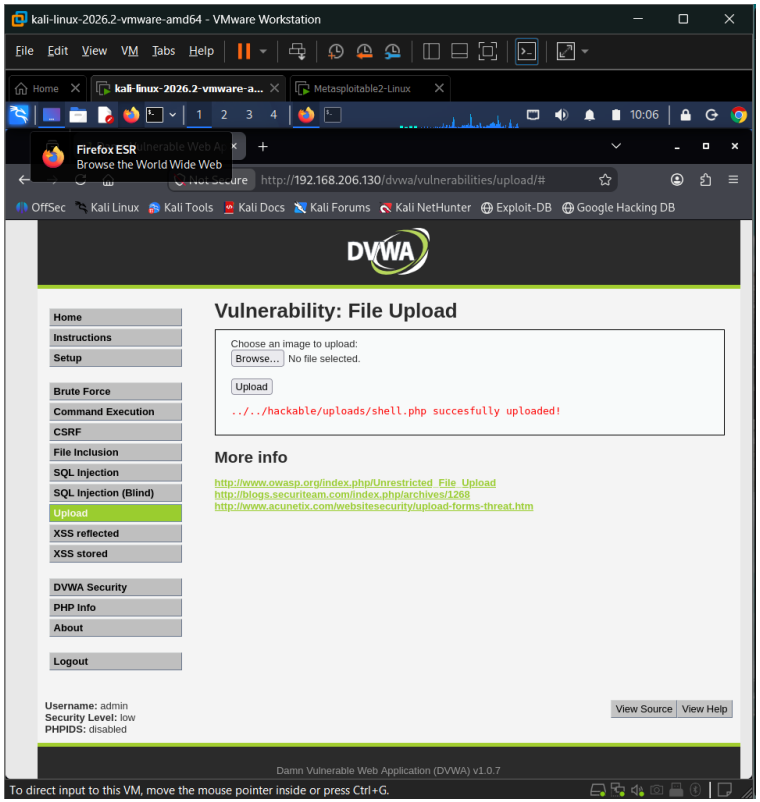


Figure 14: shell.php uploaded successfully at Low

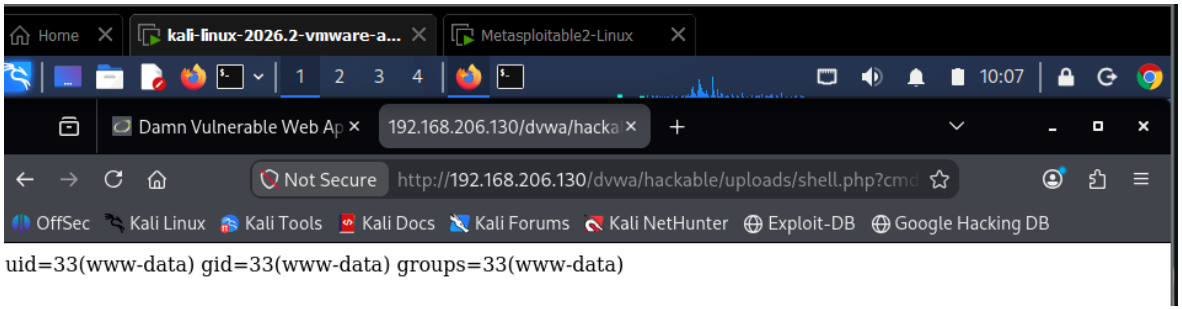


Figure 15: cmd=id confirms command execution as www-data

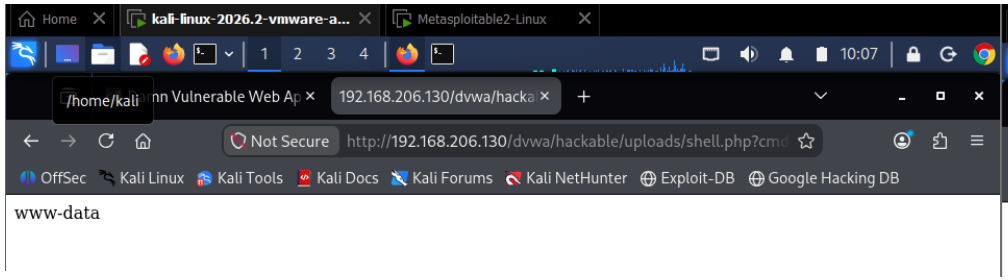


Figure 16: cmd=whoami

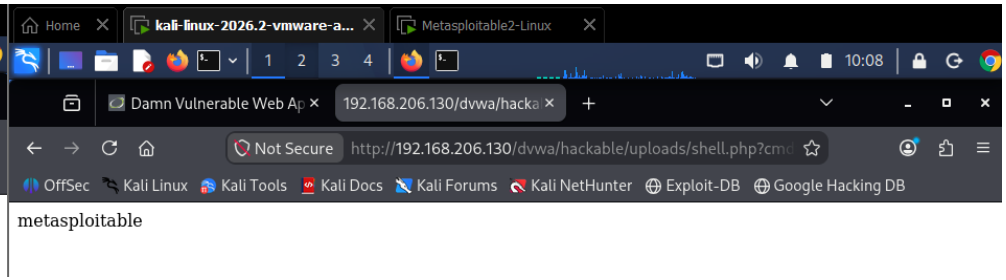


Figure 17: cmd=hostname confirms the host is “metasploitable”

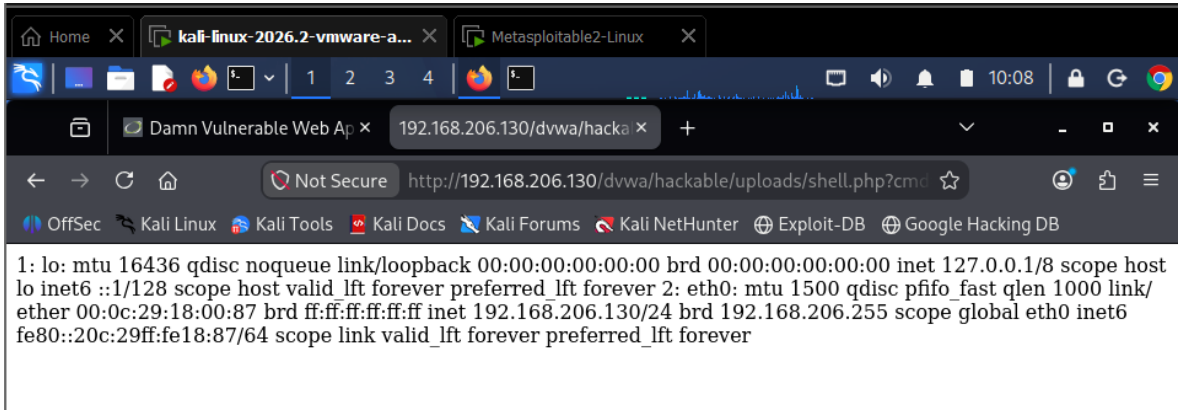


Figure 18: cmd=ip a confirms network position: 192.168.206.130/24 on eth0

Medium – Client-Supplied MIME Type, Bypassed via Burp

Lesson: the MIME type in a multipart upload is supplied entirely by the client and can be rewritten in an intercepting proxy before the server ever evaluates it. A check based on a client-supplied value is not a real validation control.

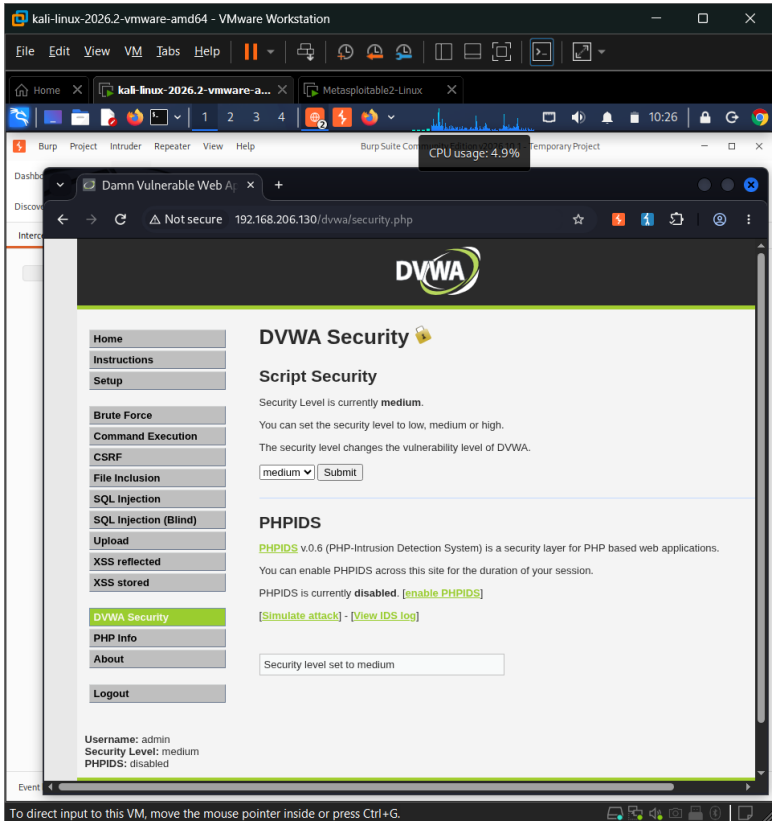


Figure 19: Security level confirmed: Medium

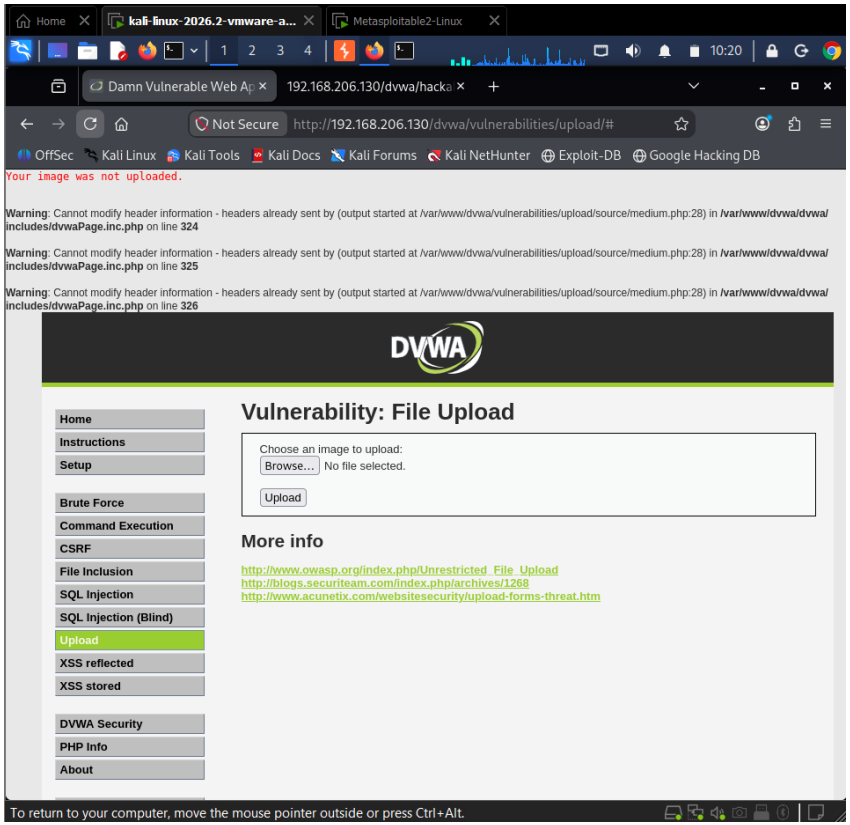


Figure 20: Unmodified shell.php upload correctly rejected: “Your image was not uploaded.”

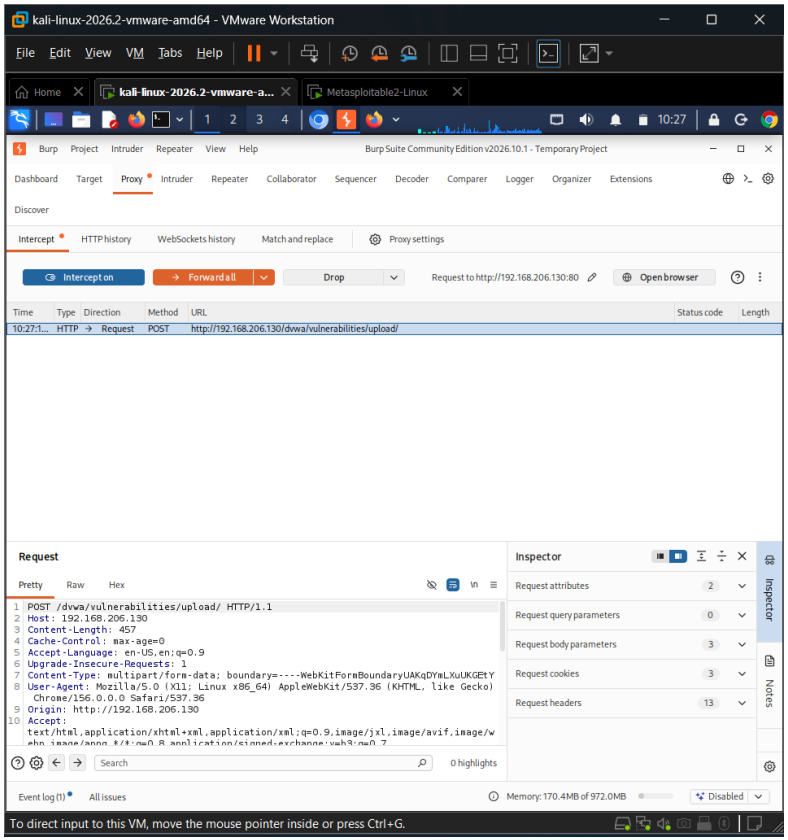


Figure 21: Burp intercepts the correct POST request to /vulnerabilities/upload/

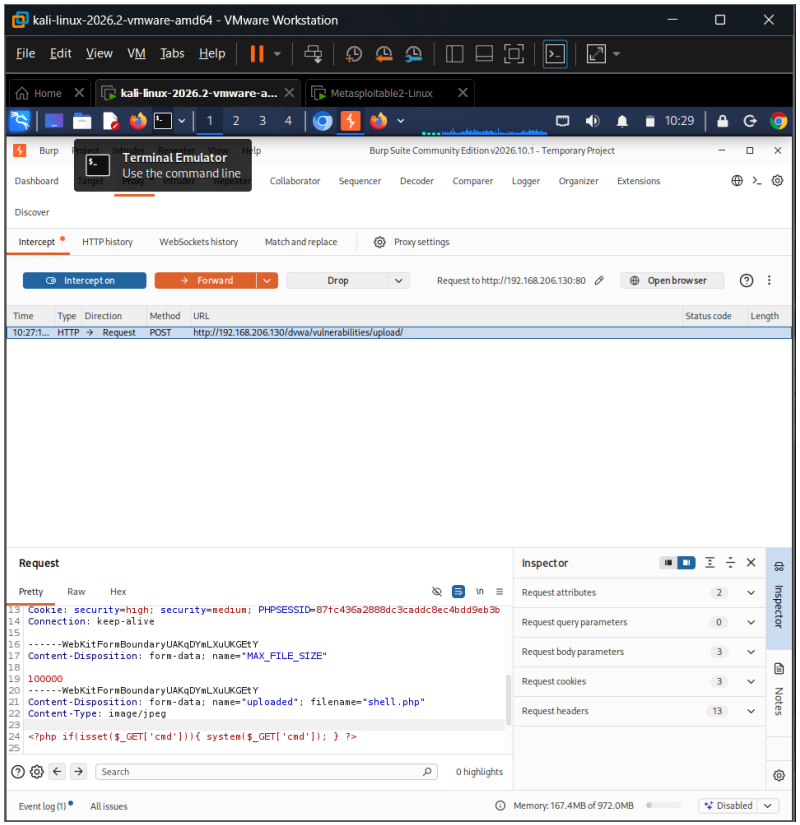


Figure 22: Content-Type edited from application/x-php to image/jpeg before forwarding

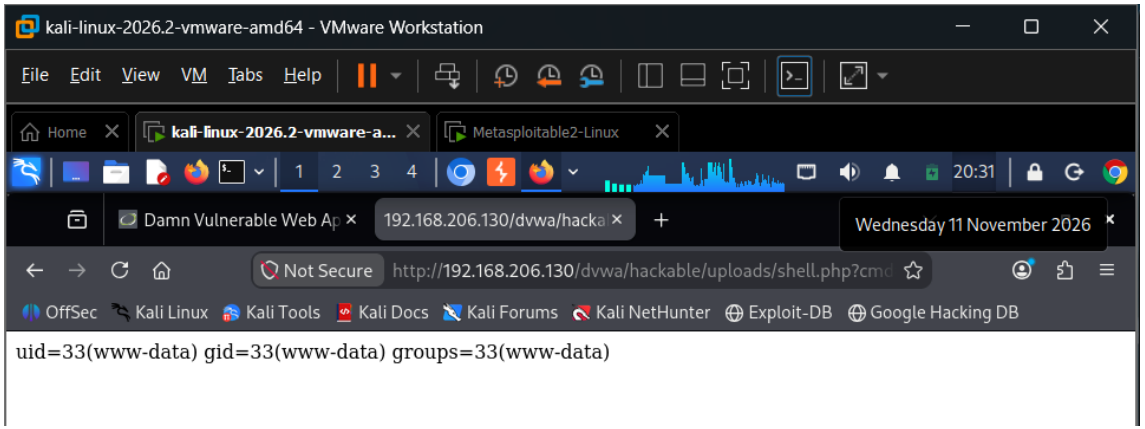


Figure 23: Forwarded request succeeds; cmd=id confirms execution again

High – Content/Extension Check, Bypassed With a Polyglot, Then Chained to Real Execution

A genuine small JPEG was generated, resized, and had the PHP payload appended after its end-of-image marker, then verified to still be a valid image before upload.

```
identify small.jpg                    # must still report dimensions
image.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1×1, segment length 16, baseline, precision 8,
 200×200, components 3
small.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1×1, segment length 16, baseline, precision 8,
 50×50, components 3
small.jpg JPEG 50×50 50×50+0+0 8-bit sRGB 353B 0.000u 0:00.000
```

Figure 24: Polyglot verified: file/identify confirm it is still a valid 50x50 JPEG after the PHP was appended

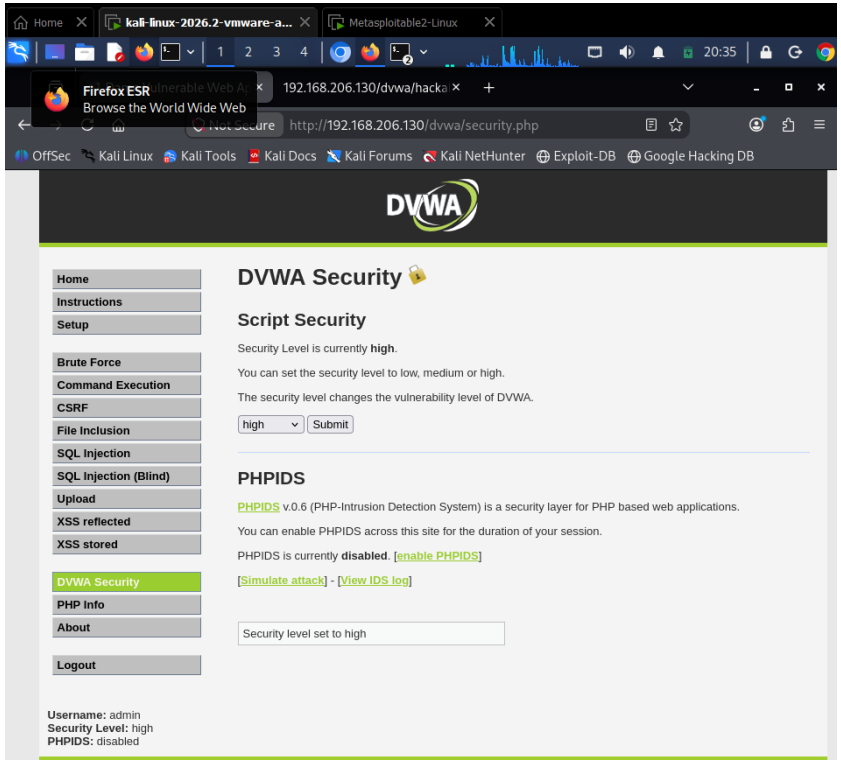


Figure 25: Security level confirmed: High

Key finding, verified against source code: the High-level check in this DVWA build performs **no content inspection whatsoever**. Its actual logic (pulled via “View Source”) is:

```
if (($uploaded_ext == "jpg" || $uploaded_ext == "JPG" ||<br/>    $uploaded_ext == "jpeg" || $uploaded_ext == "JPEG")
&amp;&amp;<br/>  ($uploaded_size &lt; 100000)) {
```

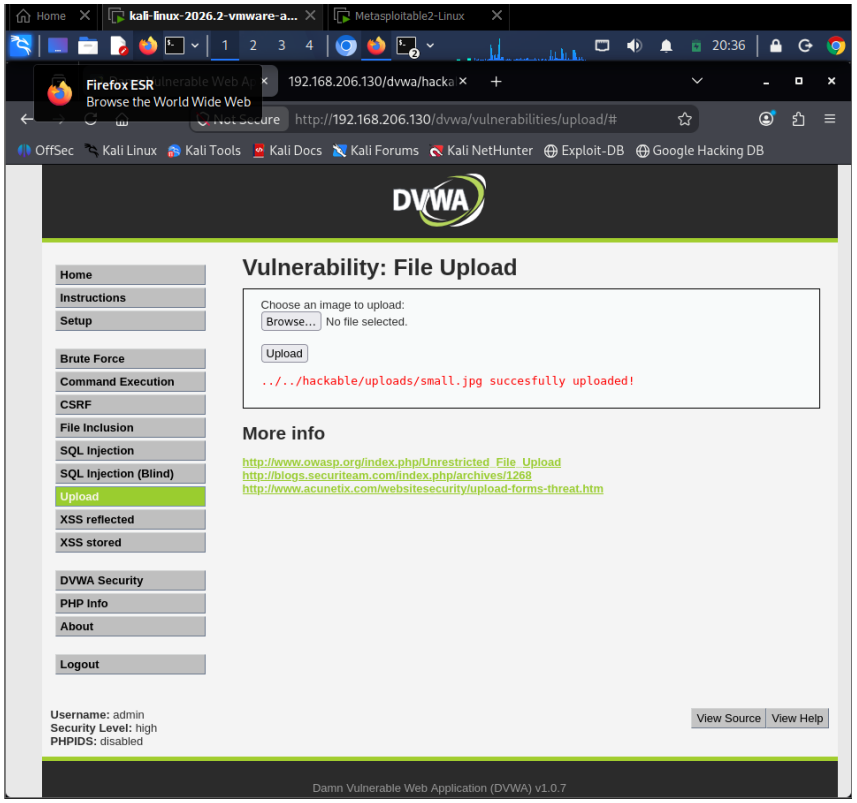


Figure 26: Polyglot uploaded successfully, unmodified filename

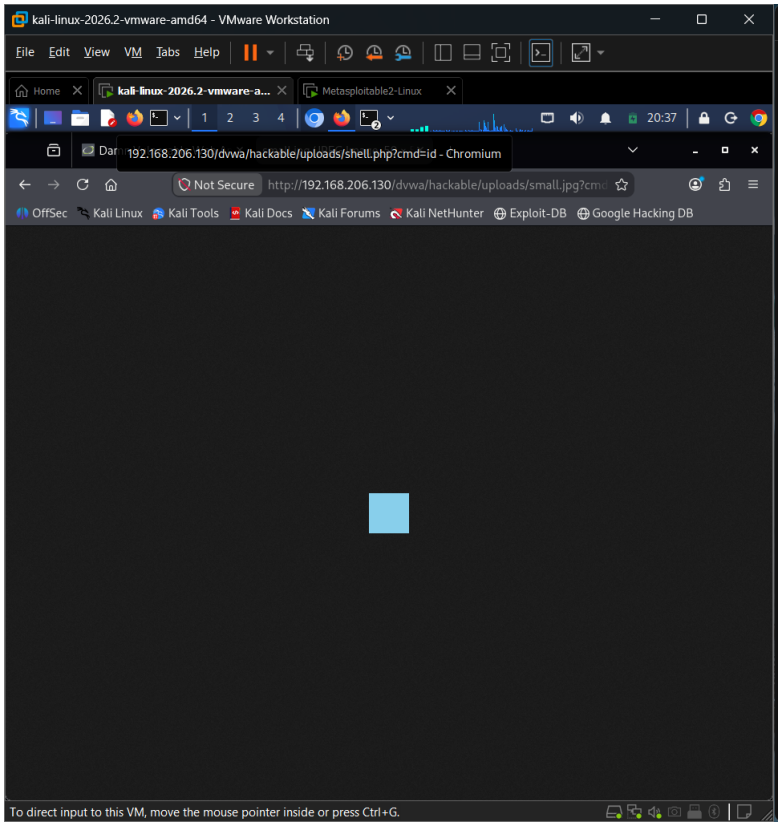


Figure 27: Direct request to the polyglot renders as an image – does not execute

```
move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path);<br/>}
```

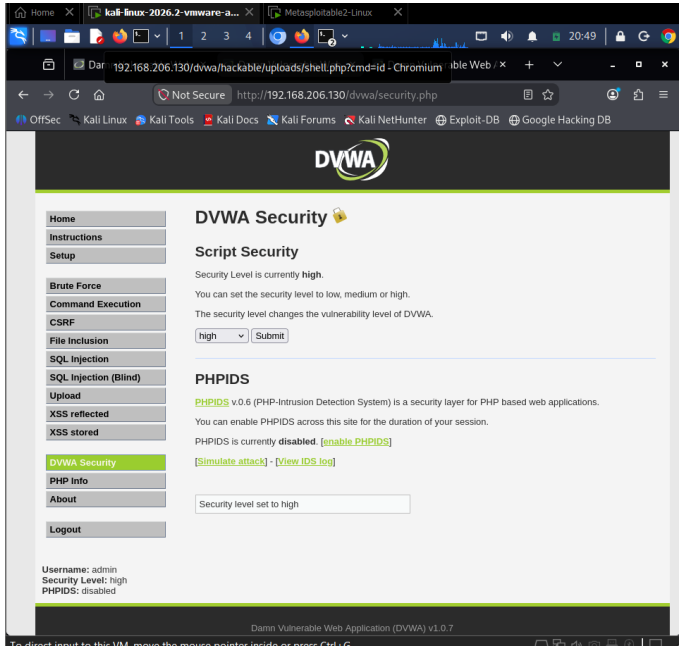


Figure 28: High-level security confirmed immediately before pulling source

This is a pure extension-and-size whitelist – not the content check the handout's description implies. The polyglot passed because it was named `small.jpg` and was under 100,000 bytes, not because a magic-byte check was defeated. Direct execution still failed, because Apache only routes .php-extension requests to the PHP interpreter; a .jpg file is served as a static image regardless of what bytes it contains.

Chaining to Real Code Execution via File Inclusion (LFI)

To demonstrate that the upload weakness is genuinely exploitable (not merely a harmless bypass of one check), the uploaded polyglot was chained with DVWA's own File Inclusion module, which uses PHP's `include()` and will parse any file as PHP regardless of its extension.

This result is reported precisely: the Upload module's own validation at High was bypassed by the polyglot, but code execution required chaining a **second, independent vulnerability** (File Inclusion) to actually interpret the file as PHP. This demonstrates defense-in-depth working *partially* – the content/extension gap was real, but a second layer (extension-based script dispatch) briefly held until a separate flaw was used to defeat it too.

Substitute “Impossible-Equivalent” Analysis

This installed DVWA build has no Impossible level. The dropdown was checked directly to confirm this rather than assumed:

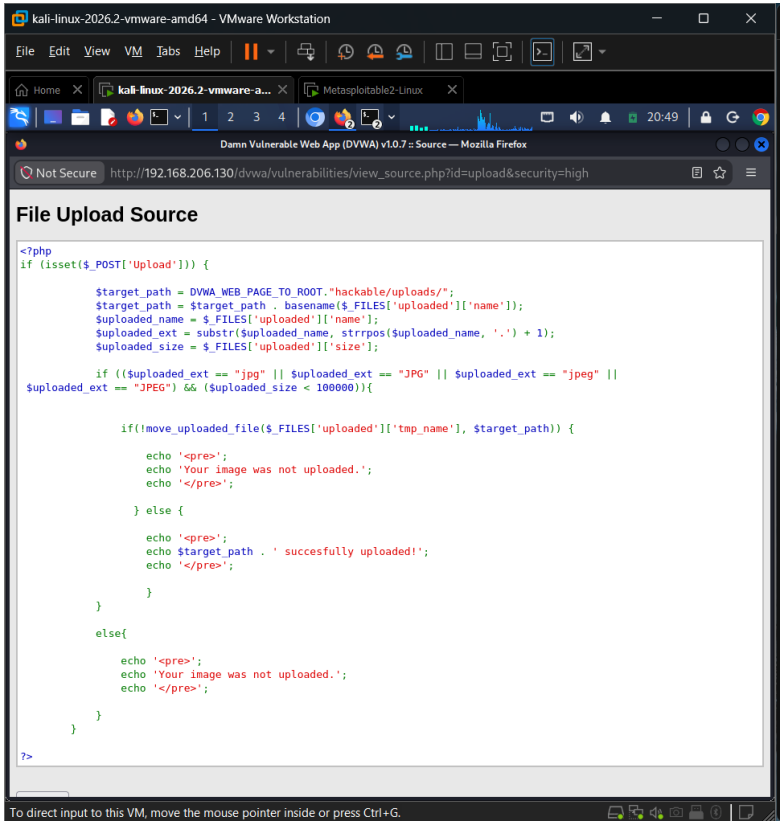


Figure 29: High-level upload source code, pulled via View Source

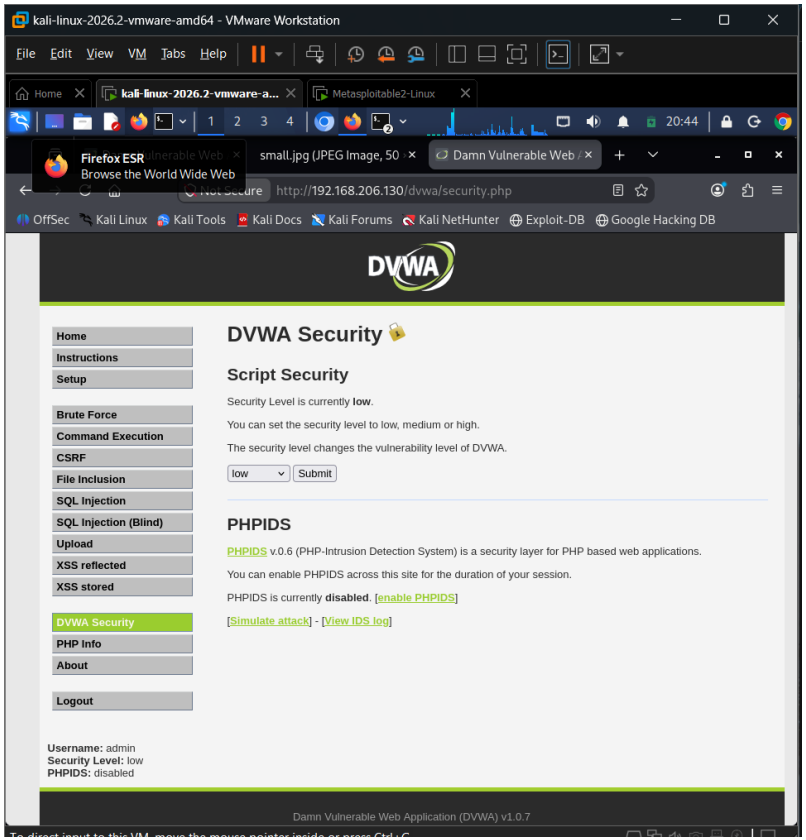


Figure 30: Security level temporarily set to Low to test the File Inclusion module directly

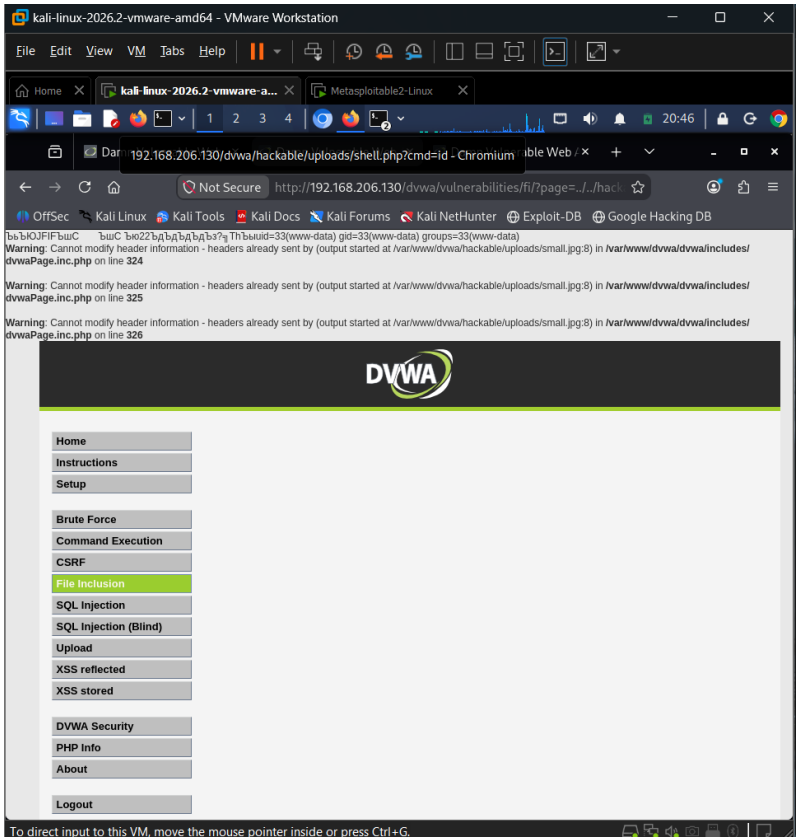


Figure 31: LFI include of the polyglot executes the embedded PHP – uid=33(www-data) confirmed, mixed with raw JPEG header bytes

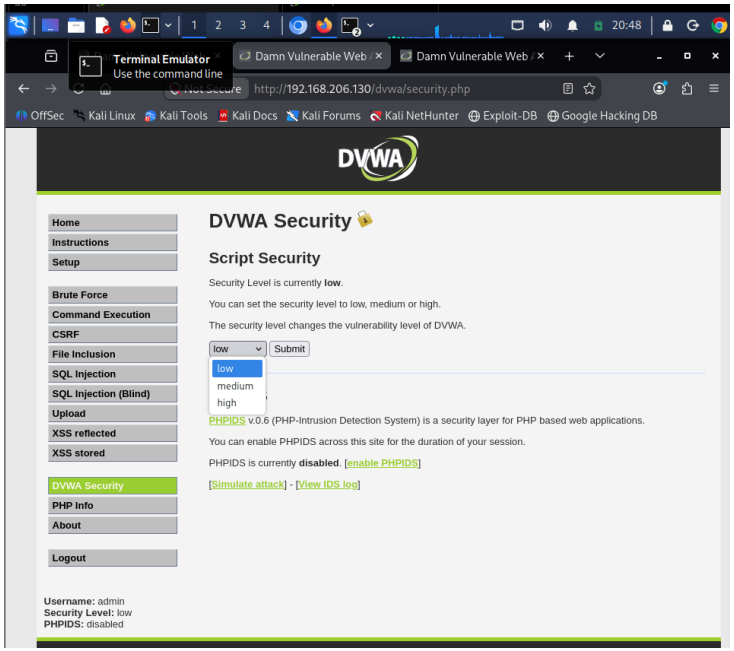


Figure 32: DVWA Security dropdown fully expanded: only low / medium / high exist

Comparing the High-level source above against what a correct implementation requires:

Control	High (as implemented)	What's missing
Content validation	None – extension string match only	Should call `getimagesize()` or equivalent to confirm the file is genuinely an image
Filename handling	Uses the attacker-supplied name verbatim (`basename($_FILES['uploaded']['name'])`)	Should generate a random server-side filename so the stored file cannot be requested by a predictable name
Storage location	Inside the web root (`hackable/uploads/`)	Should be outside the web root, or in a directory with script execution explicitly denied
Serving behavior	Served directly; browser renders or executes based on content	Should be served with `Content-Disposition: attachment`

The storage/execution-denial gap is the single most important one: it is the one control that would have defeated **every** bypass demonstrated in this report, including the polyglot and the File-Inclusion chain, because there would be nothing for `include()` or the PHP interpreter to execute even if a malicious file reached the directory.

Part B – Summary of Confirmed Findings

- **Low:** unrestricted upload → immediate remote code execution as www-data.
- **Medium:** client-supplied Content-Type → trivially bypassed via an intercepting proxy.
- **High:** extension/size whitelist with no real content check → bypassed with a genuine polyglot; direct execution blocked by extension-based interpreter dispatch, but full remote code execution achieved by chaining the upload with a separate File Inclusion vulnerability.
- **Impossible:** not present in this DVWA build (v1.0.7); a substitute gap analysis was produced directly from the High-level source code.
- Outdated Apache (2.2.8, released 2008) and PHP (5.2.4, released 2008) confirmed across multiple independent tools (curl, whatweb, nikto).

Part C – Remediation Brief (Non-Technical)

For:	Management / Non-technical stakeholders
Subject:	Critical weakness found in the file upload feature

What We Found

The web application's file upload feature does not properly check what people are allowed to submit. During testing, our team was able to upload a disguised file through the upload form and get the server to run it as a program – giving us the same level of access to the server that the website's own software has.

This is one of the most serious types of weakness a web application can have, because it does not require tricking a user or stealing a password. Anyone who can reach the upload form can potentially do this.

Why It Matters

Once a file like this runs on the server, an attacker can:

- Read configuration files, which often contain database passwords
- View, change, or delete data stored in the application's database
- Use the server as a stepping stone to reach other systems on the same network
- Leave a permanent foothold that keeps working even after the original weakness is noticed, unless it is specifically found and removed

We confirmed this is possible, measured how far it goes, and tested it against three different levels of built-in protection in the test environment. The weakest protection level offered no resistance at all. A protection that only checks the file type that the browser claims – which is one of the three protections we tested – can be defeated because that claim is something the person uploading controls and can simply change before it reaches the server. A stronger protection that checks the uploaded content more carefully still let a disguised file through, because it did not verify the content closely enough and because the storage location still allowed certain file types to run as programs.

What Needs to Change

- **Check the actual content of every uploaded file on the server**, not just its declared type or the end of its filename. The server should confirm a file really is what it claims to be before accepting it.
- **Never trust the filename the user provides.** Store uploaded files under a new, randomly generated name so an attacker cannot predict or request their file directly.
- **Store uploaded files somewhere the server will never run as a program**, regardless of what the file contains or is named. This single change would have stopped every method we tested, even the ones that got past the content checks.
- **Make browsers download files** instead of running or displaying them when a user opens an uploaded file's link directly.
- **Limit the size and rate of uploads**, and log any file whose content doesn't match the type it claims to be, so unusual activity is visible instead of silent.

Priority

This should be treated as an **urgent fix**, not routine maintenance. The underlying weakness gives an outside party the same level of control over the server as the application itself has – there is no intermediate step an attacker needs to take first.

References

- OWASP – Unrestricted File Upload: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- PayloadsAllTheThings – Upload Insecure Files: https://github.com/swisskyrepo/PayloadsAllTheThings
- CWE-434 – Unrestricted Upload of File with Dangerous Type: https://cwe.mitre.org/data/definitions/434.html

All evidence files, screenshots, the web shell, and the polyglot used in this report are published at: https://github.com/the-raven-zn/dfcs-bootcamp/tree/main/s09-recon-and-upload